This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TC.2018.2842050, IEEE Transactions on Computers



# Fast In-Place Suffix Sorting on a Multicore Computer

Bin Lao, Ge Nong, *Senior Member, IEEE,* Wai Hong Chan, and Jing Yi Xie

**Abstract**—Sorting all suffixes of an input string $X$ will produce the suffix array that is a fundamental data structure for full-text search on $X$. To utilize the parallel computing power of a multicore machine with shared memory, this article designs a fast linear-time and in-place parallel algorithm called pSACAK, for sorting the suffixes of an input string with a constant alphabet. This algorithm is a parallel variant of the sequential suffix sorting algorithm SACAK which improved the linear-time SAIS to be in-place for constant alphabets, and hence requires only a workspace of $\mathcal{O}(K)$ for alphabet size $K$. While our recent work has successfully designed the parallel variant of SAIS on a multicore machine, it remains a challenge to parallelize SACAK due to the strong data dependencies caused by the in-place constraint. A number of new techniques are proposed here to overcome the difficulties for designing pSACAK from the sequential SACAK. An experimental study is conducted to evaluate the performance of pSACAK versus other existing parallel suffix sorting algorithms. Our experimental results show that pSACAK is the most time and space efficient among all in comparison. To the best of our knowledge, pSACAK is the only linear-time and in-place parallel suffix sorting algorithm for constant alphabets reported so far.

**Index Terms**—Suffix sorting, parallel algorithm, linear-time, in-place.

## 1 Introduction

SUFFIX sorting for constant alphabets is a key problem to many applications in string processing [1], data compression [2], genome alignment [3] and etc. Sorting all suffixes of an input string $X$ will produce the suffix array (SA) [4] that is one of the fundamental data structures for full-text search on $X$. Thus the existing suffix sorting algorithms are also known as the suffix array construction algorithms (SACAs). As sorting suffixes efficiently both in time and space is critical to the usage of the suffix array in practice, many researches have been devoted to the sequential suffix sorting algorithms on different computing models [5], [6], [7], [8]. In order to achieve high performance by fully utilizing the parallel computing power of multicore CPUs and GPUs in a modern machine, parallel [9], [10], [11], [12] and hybrid [13], [14] algorithms have emerged as a rising research interest.

From the comprehensive survey [15] up to 2007, the main species of SACAs were characterized as three classes: prefix doubling, recursive and induced copying (sorting). In the past decade, the induced sorting (IS) method has been well studied to design many linear-time SACAs (such as SAIS [5] and SACAK [6]), which deliver the best performance results both in theory and practice [14], [16]. The key for these IS algorithms to achieve linear time is recursively inducing the order of all suffixes from a sorted subset of suffixes in a divide-and-conquer manner, where the subset's cardinality is at most half of the input size at each recursion level. While the IS method in SAIS has been successfully parallelized in our recent work [12], it remains an open problem to design an "in-place" parallel IS algorithm for better performance both in time and space. In this article, a space is measured in $\log n$ bits if not specified explicitly, as commonly adopted in the literatures for SACAs, e.g., [17], [18]. In [18], a SACA is considered to be "in-place" if it runs in $\mathcal{O}(1)$ workspace, where the "workspace" is the space requirement excluding the input string and the output suffix array.

While high throughputs over 30 MiB/s have been reported for suffix sorting in a GPU [10], [11], the input is scaled up to only around 100 MiB limited by the GB-range high bandwidth memory inside the GPU. However, the internal memory capacity of a modern computer has grown beyond the TB range, and this makes it feasible for sorting the suffixes of a "big" input in the internal memory. Meanwhile, the parallel computing power of a modern machine is growing rapidly while the number of cores on a CPU is consistently increased. Currently, over 20 cores per CPU has been commercial available and more cores will be delivered in the future. In order to fully utilize the rapid growing parallel computing power of a modern multicore machine, we studied the problem of parallelizing the IS method to build a "big" suffix array in the internal memory using the multithreading capability of such a machine in [12]. The experiment therein (Section 1) evaluates the performance margin for improving the IS method by parallelism via simultaneously running 1, 2, 4 and 8 processes of SAIS and SACAK programs[1] with the same input on a 12-core machine with shared memory. The time results show that the sequential SACAK runs 25% faster than the sequential

- *B. Lao and G. Nong are with the Department of Computer Science, Sun Yat-sen University, Guangzhou, China. E-mail: laobin@mail3.sysu.edu.cn, issng@mail.sysu.edu.cn.*
- *W. H. Chan is with the Department of Mathematics and Information Technology, The Education University of Hong Kong, Tai Po, Hong Kong. E-mail: waihchan@eduhk.hk.*
- *J. Y. Xie is with the Key Laboratory of Machine Intelligence and Advanced Computing (Sun Yat-sen University), Ministry of Education, Guangzhou, China. E-mail: xiejy26@mail2.sysu.edu.cn.*

*Corresponding author: G. Nong and W. H. Chan.*

1. Our source codes for SAIS and SACAK are publicly available at http://code.google.com/p/ge-nong/.

0018-9340 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TC.2018.2842050, IEEE Transactions on Computers



SAIS on average, and both their throughputs increase nearly linear to the number of processes. This indicates that a single process of each sequential program only consumes a small amount of the multicore machine's computing resources. Since the performance gain from parallelizing the IS method in SAIS has been confirmed in [12], it is expected to get a more competitive result from a parallelized SACAK on the same computing model. This motivates our study here to solve the problems for paralleling the sequential SACAK on a multicore machine with shared memory.

In recent works on the shared memory architecture, there are four state-of-the-art parallel SACAs that run fast in practice. Specifically, the Problem Based Benchmark Suite (PBBS) [19] includes two parallel implementations of the prefix doubling [20] and the DC3 [21] algorithms, but neither of them runs in-place. Moreover, a hybrid algorithm is proposed in [14], using induced copying/sorting method with the prefix doubling algorithm as a subroutine. Although it is reported to be more time and space efficient than the parallel variants of libdivsufsort, DC3, prefix doubling and etc, its workspace is input dependent. Very recently, we designed the parallel algorithm pSAIS [12] by parallelizing each step of the sequential linear-time SAIS algorithm. With a combined use of multithreading and pipelining, the inducing process in pSAIS is accelerated by utilizing the machine's parallel computing power, while only a negligible amount of additional space is required due to the read/write buffers for pipelining the induced sorting process. The workspace of SAIS is linear to the input size, so is that of pSAIS. In other words, pSAIS is linear-time but not in-place. The algorithm pSACAK (parallel SACAK) we present here is a parallel variant of the sequential suffix sorting algorithm SACAK. For an input of alphabet size $K$, SACAK is linear-time and in-place, i.e, uses $\mathcal{O}(n)$ time and $\mathcal{O}(K)$ workspace. Our algorithm pSACAK inherits the linear-time and in-place merits of SACAK for constant alphabets, but runs much faster by utilizing the parallelism of multiple cores.

The contribution of this article consists of two aspects:

- We design an in-place pipelined induced sorting procedure using multiple threads for parallelism. The original IS process is divided into three stages to be pipelined by threads for better memory access throughput using only $\mathcal{O}(1)$ workspace.
- We implement pSACAK in C++ with Intel Cilk Plus [22] for parallel programming. This program is evaluated by experiments on a set of typical corpora. Comparing with other parallel SACAs, our program yields a competitive time and space performance on a multicore machine.

In the rest of this article, we present the algorithm framework of pSACAK in Section 2 and explain the detailed designs in Section 3-5. The experimental study for performance evaluation is conducted in Section 6, and the conclusions are given in Section 7.

## 2 Algorithm Framework

For presentation convenience, we first introduce some basic notations commonly used in our algorithm. Let $X[0, n-1]$ be an input string with $n$ characters from a constant alphabet $A[0, K-1]$, where $K = \mathcal{O}(1)$ and $X[n-1]$ is the lexicographically smallest character in $X$ called the "sentinel". That is, the size-$n$ input string $X[0, n-1]$ consists of $n-1$ characters $X[0, n-2]$ and a sentinel $X[n-1]$. The suffix array $SA$ of $X$ is a size-$n$ integer array such that $SA[i] = j$ stores the starting position of the $i$-th suffix $\mathsf{suf}(X, j)$ in ascending lexicographical order, where $i, j \in [0, n-1]$ and $\mathsf{suf}(X, j)$ denotes the suffix $X[j, n-1]$ in $X$. Considering the example in Fig. 1, the input string $X[0, 16]$ ="mmiissiissiippie$" with a sentinel $X[16]$ ='$' is used to illustrate the concept of suffix/character type. Specifically, a suffix $\mathsf{suf}(X, i)$ and its head character $X[i]$ are both S-type if (1) $i = n-1$ or (2) $X[i] < X[i+1]$ or (3) $X[i] = X[i+1]$ and $X[i+1]$ is S-type; otherwise, they are both L-type. Moreover, $\mathsf{suf}(X, i)$ and $X[i]$ are classified as leftmost S-type (LMS) if $X[i]$ is S-type and $X[i-1]$ is L-type for $i > 1$, and the sentinel itself is also LMS. The row "$t$" gives the type of each suffix/character with 0 for L-type and 1 for S-type, respectively. As the row "LMS" marks each LMS-character, a substring $X[i, j]$ is called an LMS-substring if (1) $i = j = n-1$ or (2) $i < j$, both $X[i]$ and $X[j]$ are LMS with no other LMS-character between them. There are four LMS-substrings {iissi, iissi, iippie$, $} in the example. At the row "bucket", all the sorted suffixes in $SA$ can be partitioned into multiple consecutive buckets by their head characters, and there are 6 buckets marked at their start positions with head characters {$, e, i, m, p, s}. Moreover, the first and the last items of a bucket are called the "start" and the "end" of the bucket, respectively.

| Index: | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $X$: | m | m | i | i | s | s | i | i | s | s | i | i | p | p | i | e | $ |
| $t$: | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 0 | 0 | 1 |
| LMS: | | | * | | | | * | | | | * | | | | | | * |
| $SA$: | 16 | 15 | 14 | 10 | 6 | 2 | 11 | 7 | 3 | 1 | 0 | 13 | 12 | 9 | 5 | 8 | 4 |
| Bucket: | $ | e | i | | | | | | | m | | p | | s | | | |

Fig. 1. An example of the basic notations for our algorithm.

The algorithmic framework of pSACAK is shown in Algorithm 1. pSACAK is a parallel variant of the sequential algorithm SACAK, in terms of that it keeps the recursive framework of SACAK by only parallelizing each step. Due to the space limit, we can not explain SACAK in detail here, interested readers may refer to [6] for the details. Because SACAK is linear-time and in-place, so does pSACAK, but the latter has a speedup over the former by utilizing the machine's parallel computing power. pSACAK first sorts all the LMS-substrings of $X$ in lines 4 and 6, then names each LMS-substring with an integer to produce a reduced string $X_1$ in line 7, so as to compute the suffix array $SA_1$ of $X_1$ recursively in lines 8-11 and induce $SA$ from $SA_1$ in lines 12-16.

The advantages of SACAK over SAIS are inherited by their parallel variants, saying that pSACAK takes advantages over pSAIS as follows.

- Type array $t$ is not required any more, and bucket counter array $bkt$ is needed only at level 0 (i.e., the top recursion level).
- The pipelined induced sorting methods for level 0

0018-9340 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TC.2018.2842050, IEEE Transactions on Computers



**Algorithm 1:** The framework of pSACAK.

**Input:** $X$: input string;
$K$: alphabet size of $X$;
$n$: size of $X$;
$level$: recursion level;
**Output:** $SA$: suffix array of $X$;

1 **Function** pSACAK($X, SA, K, n, level$)
/* Reduction Phase */
2 **if** $level = 0$ **then**
3 Allocate an array of $K$ integers for $bkt$;
4 Pipelined induced sort all the LMS-substrings of $X$, using $bkt$ for bucket counters;
5 **else**
6 Pipelined induced sort all the LMS-substrings of $X$, reusing the start or the end of each bucket as its counter;
/* $SA$ is reused for storing $X_1$ and $SA_1$. */
7 Parallelized compute the lexicographic names for the sorted LMS-substrings to produce $X_1$;
/* Recursion Phase */
8 **if** $K_1 = n_1$ **then** // Each character in $X_1$ is unique.
9 Directly compute $SA_1$ from $X_1$;
10 **else**
11 pSACAK($X_1, SA_1, K_1, n_1, level + 1$);
/* Induction Phase */
12 **if** $level = 0$ **then**
13 Pipelined induced sort $SA$ from $SA_1$, using $bkt$ for bucket counters;
14 Free the space allocated for $bkt$;
15 **else**
16 Pipelined induced sort $SA$ from $SA_1$, reusing the start or the end of each bucket as its counter;
17 **return**

and deeper levels (i.e., levels 1, 2, and thereafter) are distinct.

- Each sorted LMS-substring is named by its "s-rank" or "se-rank" in $X_1$. According to Section 5 of [6], the s-rank of $X_1[i]$ is the number of characters in $X_1$ smaller than $X_1[i]$, and the se-rank of $X_1[i]$ is the number of characters in $X_1$ smaller than or equal to $X_1[i]$ (excluding $X_1[i]$ itself). Hence, the string $X_1$ produced in pSACAK may be different from that in pSAIS, although both are of the same length $n_1$.

The parallel strategy for each step of pSACAK in Algorithm 1 is summarized as follows. Line 7 can be parallelized by dividing the sorted LMS-substrings into blocks and assigning each block to a thread for naming in parallel. The parallel naming method without type array $t$ is explained in Section 4. The most challenging part to be parallelized is induced sorting substrings and suffixes with the optimal $\mathcal{O}(1)$ workspace in lines 4, 6, 13 and 16, where the in-place constraint causes strong data dependencies to be solved for parallelization. We proceed to describe the new challenges and our solutions in the algorithm's design.

## 3 Sorting Suffixes at Level 0

At level 0, the method for SACAK to induce the order of suffixes from the sorted LMS-substrings performs three steps in sequence using $SA$ and a bucket counter array $bkt$ of space $\mathcal{O}(K) = \mathcal{O}(1)$: (1) put each sorted LMS-suffix of $X$ into $SA$ to the end of its bucket in $SA$; (2) induce the order of all L-type suffixes of $X$ from the sorted LMS-suffixes; and (3) induce the order of all S-type suffixes of $X$ from the sorted L-type suffixes. The difficulty for turning this method into parallel is due to the heavy data dependencies in steps (2) and (3) for accessing $SA$ and $bkt$.

According to our experiments in [12], the induced sorting process in pSAIS consumes over half of the total running time and is indeed the performance bottleneck of the whole algorithm. At level 0, the pipelined schema and parallel strategy of induced sorting in pSACAK are mostly the same as that in pSAIS, where the $SA$ is evenly divided into blocks and each block is processed by three pipelined stages. Let $B_{k-1}$, $B_k$ and $B_{k+1}$ be the preceding, current and succeeding blocks of $SA$, respectively. When the inducing stage in pipeline is processing $B_k$, each stage does as follows.

1) Preparing: the preceding character of each sorted suffix in $B_{k+1}$ is read from $X$ then stored in a read buffer by multiple threads in parallel.
2) Inducing: a thread sequentially induces the order of each preceding suffix by scanning $B_k$ in $SA$, and a write buffer is used to store most of the newly sorted suffixes not destined for $B_k$ or $B_{k+1}$.
3) Updating: the newly sorted suffixes of $B_{k-1}$ in the write buffer are written to $SA$ by multiple threads in parallel.

Different from pSAIS where an $n$-bit type array $t$ is available at each level for induced sorting suffixes, there is no $t$ in pSACAK, neither implicitly nor explicitly. Under this constraint, there are two problems to be solved for parallelization: (1) how to determine the type of a suffix during pipelined induced sorting; (2) how to find the LMS-substrings of $X$ in the initialization for sorting process. The parallelized solutions for these two problems are given below, respectively.

- Solution 1: Property 3.1 is employed to determine the type of a suffix, when a thread is sorting a block (i.e., $B_k$) of $SA$. Herein, we call an item in $SA$ to be "non-empty" if it is storing the index of a suffix or being reused as a bucket counter; otherwise, it is "empty" (marked by the least negative integer).
- Solution 2: Utilizing the type pattern $S^+L^+S$ of an LMS-substring, which consists of one or more S-type characters, one or more L-type characters and a single S-type character, a two-stage algorithm is designed for retrieving LMS-substrings from $X$ in parallel as follows: (1) $X$ is evenly divided into blocks and each block is traversed once from left to right by a thread in parallel, so as to find out every "target" L-type character in $X$, where a target L-type character is lexicographically greater than its succeeding; (2) if any target L-type character $X[i]$ is found in the block, its succeeding character $X[i+1]$ is a potential LMS-character. Therefore, the thread

0018-9340 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TC.2018.2842050, IEEE Transactions on Computers



verifies the type of $X[i+1]$ by continuing the traverse till it meets another character $X[j] \neq X[i+1]$ or the sentinel. If $X[j] > X[i+1]$ or $X[j]$ is the sentinel, it means that $X[i+1]$ must be an LMS-character and also the start of the succeeding LMS-substring. Thus each LMS-substring can be identified on-the-fly in this way.

**Property 3.1.** *[6] At level 0, when induced sorting the S-type suffixes of $X$ from the sorted L-type suffixes, for each non-empty $SA[i]$ and $j = SA[i] - 1$, $\mathsf{suf}(X, j)$ is S-type if and only if (1) $X[j] < X[SA[i]]$ or (2) $X[j] = X[SA[i]]$ and $bkt[X[j]] < i$.*

Solution 2 may not be load balanced for each thread, since the number of LMS-substrings in each block is input-dependent and may vary much. In this case, we divide the input string into a set of small blocks, and each thread takes one out from the set to process until the set is empty. Now, the load for each thread is likely even. Such a load balancing method was initially used in [12].

## 4 Naming Sorted Substrings

SACAK calculates the name for each sorted LMS-substring of $X$ in two sequential steps: (1) name each LMS-substring by the start of its bucket in $SA_1$ to get an interim string; (2) replace each S-type character in the interim string with the end of its bucket in $SA_1$ to get the new string $X_1$. As a result, the start and the end of each bucket in $SA_1$ can be reused on-the-fly as the bucket counter array required for computing $SA_1$. However, reusing space in this way will increase the data dependency for accessing $SA_1$ when sorting suffixes at deeper levels in Section 5, resulting in more difficulties for the parallelization of SACAK.

In pSACAK, the same pipelined IS method is employed to sort both substrings and suffixes, only the initializations of sorting processes are different (see SACAK [6] for the details). Given that all the substrings of $X$ have been sorted, the reduced string $X_1$ (which is also the input string $X$ at the next recursion level) is produced in parallel by three steps as follows.

1) The sorted substrings are evenly divided into blocks, and each block computes its difference bit vector independently by a thread in parallel. Each thread scans its block to retrieve each substring from $X$. If any substring is different from its preceding one in the block, the difference bit for this substring is set as 1, or otherwise 0. After the scan, we get the difference bit vector for all the sorted substrings.
2) Using the result of step 1, each thread names every substring within its block in parallel, by the start position of the substring's bucket in $SA_1$. This results in an interim reduced string denoted by $Z_1$, where each character points to the start of its bucket in $SA_1$.
3) $Z_1$ is sequentially scanned once from right to left. Now the type of each character in $Z_1$ can be determined on-the-fly, and each S-type character is replaced with the end position of its bucket in $SA_1$. This results in the new string $X_1$. This step can be parallelized by multiple threads in a blockwise manner for further performance improvement.

Step 1 can be load balanced for each thread using the same blockwise manner as pSAIS, and step 2 is naturally load balanced. In the new string $X_1$, each L-type or S-type character itself is also its s-rank or se-rank in $X_1$, respectively. Since $X_1$ is also the input string $X$ at the next recursion level, readers may refer to the row "$X$" in Fig. 4 as an example for deeper levels, where $X$ holds Property 4.1 of [6] cited as below.

**Property 4.1.** *[6] At deeper levels, each L-type or S-type character in $X$ itself also points to the start or the end of its bucket in $SA$, respectively.*

## 5 Sorting Suffixes at Deeper Levels

### 5.1 Challenges

To achieve $\mathcal{O}(1)$ workspace for the whole algorithm, there is no specific bucket counter array in the sequential SACAK for induced sorting at deeper levels. Accordingly, the start or the end item of each bucket in $SA$ is reused as its bucket counter during induced sorting. This is another key problem to be solved for the parallelization of SACAK, apart from the inherently sequential property of the IS method. Algorithm 2 shows the process for sequential induced sorting L-type suffixes at deeper levels. Therein, let $BK_D$ be the destined bucket of a newly sorted suffix, while $BK_L$ and $BK_R$ be the left and right neighboring buckets of $BK_D$, respectively. Due to the space reusing method for counters in $SA$, in two different cases, a full bucket is shifted left or right to put each suffix in the bucket at its final destination. Lines 7 and 21 are the bucket shifting triggered by these two cases:

- Case 1: If a newly sorted L-type suffix is destined for the start item of its bucket $BK_D$ in $SA$ and the destination has been "borrowed" by the largest item of the left neighboring bucket $BK_L$, then $BK_L$ is full and all its suffixes should be shifted left by one step to their correct locations in $SA$ as line 7.
- Case 2: If a newly sorted L-type suffix is destined for the end item of its bucket $BK_D$ in $SA$ and the destination is being reused as a counter by the right neighboring bucket $BK_R$, then $BK_D$ is full and all its suffixes should be shifted left by one step to their correct locations in $SA$ as line 21.

The operations in these two cases hold strong data dependencies. For a better illustration of bucket shifting, Fig. 2a is given as an example of case 1 & 2 for sorting the L-type suffixes of a sample string $X = \{9, 9, 8, 8, 13, 13, 8, 8, 13, 13, 8, 8, 11, 11, 1, 1, 0\}$ at deeper levels. Note that each character in $X$ itself also points to its bucket counter in $SA$ according to Property 4.1, and each counter reuses the start item of its bucket in $SA$. Due to the existences of reused counters, the suffixes in bold will be shifted when their buckets are fully filled, and those underlined are adjusted to their correct locations by shifting the bucket one step to the left. As the start time of a bucket shifting is not predictable, prefetch or stream instructions can not be applied trivially for reducing the mean latency of random memory accesses to accelerate the algorithm. To avoid data racing in pSACAK, we must guarantee that when a full bucket is

0018-9340 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TC.2018.2842050, IEEE Transactions on Computers



**Algorithm 2:** Sequential sorting L-type suffixes at deeper levels.

**Input:** $X$: input string;
$SA$: suffix array with sorted LMS-suffixes;
**Output:** $SA$: suffix array with sorted L-type suffixes;

```
1  Scan SA once from left to right as follows;
2  foreach SA[i] > 0 being scanned do
3    j ← SA[i] − 1;
4    if X[j] is L-type, i.e., X[j] ≥ X[j + 1] then
5      Get the start position of BK_D by c ← X[j];
6      if SA[c] is storing the largest item of BK_L then
7        Shift left all items in BK_L by one step;
8        Set SA[c] as empty;
9      if SA[c] is empty then
10       if SA[c + 1] is empty then
11         Initialize SA[c] ← −1 as a counter;
12         SA[c + 1] ← j;
13       else                                  // A size-1 bucket.
14         SA[c] ← j;
15     else          // SA[c] is being reused as a counter.
16       d ← SA[c], p ← c − d + 1;
17       if SA[p] is empty then
18         Increase the counter by 1, i.e., SA[c] − −;
19         SA[p] ← j;
20       else     // Running into the start item of BK_R.
21         Shift left all items in BK_D by one step;
22         SA[p − 1] ← j;
23 return
```

shifted by a thread, no item in the bucket is accessed by any other thread.

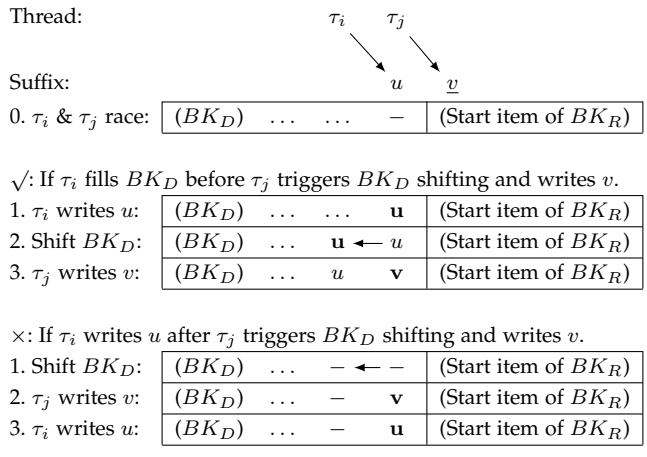
The memory accesses in Algorithm 2 are summarized as follows: (1) during suffix sorting, each item of $SA$ is sequentially read once and randomly written at most once, while each item of $X$ is randomly read at most once; (2) for the accesses to bucket counters, at most $K$ items of $SA$ are randomly read and written, at most $n$ times in all; (3) for the bucket shifting, each bucket in $SA$ can be sequentially shifted at most once. This results in at most $2n + n + 0 = 3n$ random memory accesses. In a modern machine of a memory hierarchy with caches, these random memory accesses constitute the speed bottleneck of the sequential algorithm and hence must be parallelized for acceleration.

The IS method without type array $t$ can be parallelized by a three-stage pipeline as that for level 0. However, at deeper levels, if we simply use the same pipeline with level 0, there will be two kinds of data races among the pipelined stages due to the bucket shifting, as shown in Fig. 2b and 2c. The detailed description for each data race is given below.

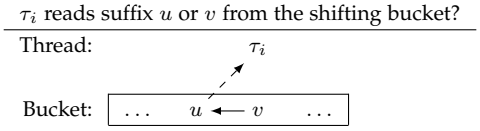
- Data race 1: $SA$ may be updated simultaneously by the thread $\tau_i$ of updating stage for $B_{k-1}$ and the thread $\tau_j$ of inducing stage for $B_k$, if $\tau_i$ writes suffix $u$ after $\tau_j$ triggers $BK_D$ shifting and writes suffix $v$, then the bucket shifting is triggered incorrectly before $BK_D$ is fully filled and produces incorrect results in $SA$ as Fig. 2b.
- Data race 2: During the bucket shifting in $SA$, the thread $\tau_i$ of preparing stage for $B_{k+1}$ may read the item within the shifting bucket, and produce uncertain result for the preceding character of suffix $u$ or $v$ as Fig. 2c.

| Index: | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $X$: | 9 | 9 | 8 | 8 | 13 | 13 | 8 | 8 | 13 | 13 | 8 | 8 | 11 | 11 | 2 | 1 | 0 |
| $t$: | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 0 | 0 | 1 |
| LMS: | | | * | | | | * | | | | * | | | | | | * |
| Case 1: | | | **14** | | | | | | | | | | | | | | |
| $SA$: | 16 | *−1* | **15** | − | − | − | 10 | 6 | 2 | − | − | − | − | − | − | − | − |
| After bucket shifting. | | | | | | | | | | | | | | | | | |
| $SA$: | 16 | **15** | *−1* | **14** | − | − | 10 | 6 | 2 | − | − | − | − | − | − | − | − |
| Case 2: | | | | | | | | | | | **0** | | | | | | |
| $SA$: | 16 | 15 | *−1* | **14** | − | − | 10 | 6 | 2 | *−2* | **1** | *−1* | **13** | *−2* | **9** | **5** | − |
| After bucket shifting. | | | | | | | | | | | | | | | | | |
| $SA$: | 16 | 15 | *−1* | **14** | − | − | 10 | 6 | 2 | 1 | 0 | *−1* | **13** | *−2* | **9** | **5** | − |

(a) Case 1 & 2




(b) Data race 1




(c) Data race 2

Fig. 2. An example of bucket shifting and data races during induced sorting L-type suffixes at deeper levels. In Fig. 2a, a solid arrow indicates an operation of induced sorting. The underlined suffixes are adjusted to their correct locations by the bucket shifting, and the other bold ones will be shifted when their buckets are fully filled. According to Property 4.1, each character in $X$ itself also points to its bucket counter in $SA$. Each counter reuses the start item of its bucket in $SA$ and is rendered in italic text. The empty items in $SA$ are marked as '$-$'. In Fig. 2b and 2c, a dashed/solid arrow indicates a read/write operation. At deeper levels, if we simply use the same pipeline with level 0, there will be two kinds of data races among the pipelined stages due to the bucket shifting.

We need a new parallel strategy to avoid these data races in the pipelined induced sorting at deeper levels. Fig. 3 shows the architecture and execution order of the new pipeline. When the inducing stage in pipeline is processing $B_k$, each stage does as follows.

1) Preparing: after the updating stage for $B_{k-1}$ is complete, the preceding character of each sorted suffix

0018-9340 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TC.2018.2842050, IEEE Transactions on Computers



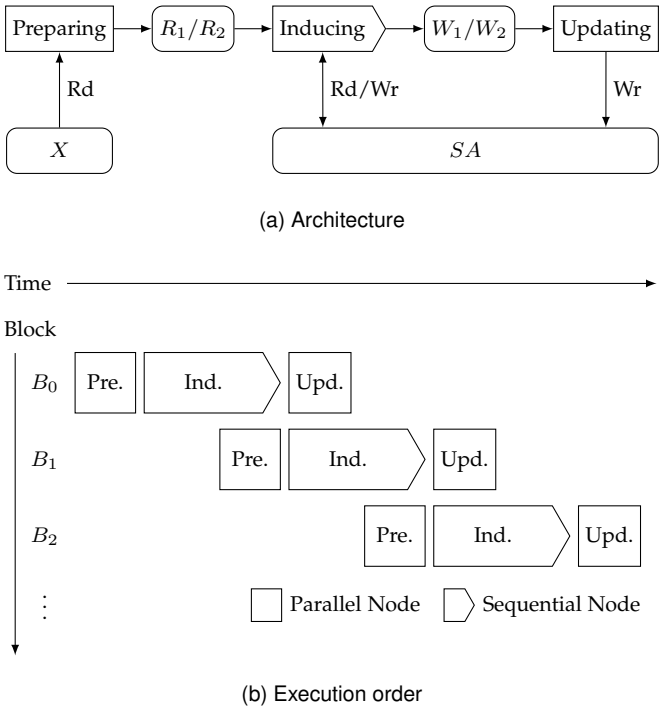



Fig. 3. Architecture and execution order of the pipeline at deeper levels. In Fig. 3a, the arrow between inducing stage and $SA$ indicates the access to a bucket counter, which is reusing the start or the end item of the bucket in $SA$. In Fig. 3b, the preparing stage for $B_2$ starts only after the updating stage for $B_0$ is complete.

in $B_{k+1}$ is read by multiple threads in parallel. And those L-type preceding suffixes are stored in a read buffer.

2) Inducing: a thread sequentially induces the order of the preceding suffix of each sorted suffix in $B_k$ by scanning the read buffer. A write buffer is used to store all of the newly sorted suffixes.
3) Updating: the suffixes in the write buffer are written to $SA$ by multiple threads in parallel. If necessary, any bucket in $SA$ can be shifted in parallel without data race.

Readers may compare the above general ideas of three pipelined stages with those for level 0 presented in Section 3. Now, we proceed to describe the new pipeline for deeper levels in detail.

### 5.2 Pipeline

Stage 1: preparing for $B_{k+1}$. In order to avoid the data races of bucket shifting in $SA$, the preparing stage for $B_{k+1}$ starts only after the updating stage for $B_{k-1}$ is complete. The task of reading the preceding characters of all the sorted suffixes currently in $B_{k+1}$ is evenly distributed to multiple threads. Then the preceding characters are read in parallel, and those L-type preceding suffixes are stored in a read buffer $R$. Two buffers $R_1$ and $R_2$ are used alternatively for $R$ to avoid access conflicts between stage 1 and 2. $X$ is read for preceding characters in this stage.

Stage 2: inducing for $B_k$. After the preparing for $B_k$ at the previous pipeline cycle, those L-type preceding suffixes of all the sorted suffixes in $B_k$ are stored in the read buffer $R$. Different from scanning $B_k$ in $SA$ at level 0, we sequentially scan $R$ to induce the order of the preceding suffix of each sorted suffix in $B_k$ at deeper levels. This change can isolate the sorting process on $R$ from any bucket shifting in $SA$, hence guarantee the inducing stage to be correct. As a result, the inducing stage for $B_k$ can be pipelined with the updating stage for $B_{k-1}$ and the preparing stage for $B_{k+1}$ without any data race.

For each newly sorted suffix, the character $chr$ and position $pos$ of the suffix's head in $X$ are read from $R$. We compute its position $idx$ in $SA$ by retrieving the bucket counter $SA[chr]$, which is reusing the start item of the bucket in $SA$. Then $SA[chr]$ is increased by one to record the head position of the bucket, that is $SA[chr]--$ as it is negative for a counter. If any newly sorted suffix is destined for $B_k$ or $B_{k+1}$ (i.e., the correct location $idx-1$ belongs to $B_k$ or $B_{k+1}$), and its preceding suffix is L-type as well, then that preceding character needs to be randomly read from $X$ and put into $R$ to be scanned later. Compared to $SA$, $B_k$ and $B_{k+1}$ are very small and hence the probability for a newly sorted suffix to be destined for $B_k$ or $B_{k+1}$ is generally small. Consequently, most of the preceding characters can be sequentially read from $R$, only a few are randomly read from $X$. All newly sorted suffixes are put into a write buffer $W$ sequentially, waiting for the updating stage for $B_k$ at the next pipeline cycle. Each suffix in $W$ is stored as a tuple $\langle idx, pos\rangle$ to denote writing $pos$ into $SA[idx]$. Two buffers $W_1$ and $W_2$ are used alternatively for $W$ to avoid access conflicts between stage 2 and 3. $SA$ is read and written for bucket counters in this stage.

Notice that each position $idx$, i.e., where the newly sorted suffix should reside in $SA$, is given by its bucket counter, and it is only sequentially read and written by this stage as depicted in the architecture. Thus any two writes to $SA$ must be for two different positions in $SA$ and hence free of conflict.

Stage 3: updating for $B_{k-1}$. The task of writing the sorted suffixes in $W$ to $SA$ is evenly distributed to multiple threads. Any bucket in $SA$ can be shifted in parallel if necessary. Notice that the suffixes in a bucket will be shifted only when that bucket is fully filled, thus the counter for a full bucket is not needed anymore, and no other suffix will be sorted into the bucket thereafter. As the suffixes destined for a same bucket may be written by different threads in parallel, we must guarantee these writes are synchronized to be done correctly before the bucket shifting. $SA$ is written for suffix updating and bucket shifting in this stage.

### 5.3 Solutions

Table 1 summarizes the strategies for resolving the data dependencies caused by bucket shifting. Recall that in line 7 of Algorithm 2, the left neighboring bucket $BK_L$ will be shifted in case 1. In pSACAK, such a bucket shifting is eliminated by initializing every bucket counter in parallel before the pipelined sorting. Thereafter, any bucket shifting must happen in case 2 only within stage 3.

For any three consecutive blocks, stage 2 and 3 have no data dependency as we are now inducing on $R$ while updating on $SA$, and the data race 1 is eliminated by synchronizing the writes of all suffixes destined for a same bucket $BK_D$ to be done correctly among the threads before

0018-9340 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TC.2018.2842050, IEEE Transactions on Computers



TABLE 1
Strategies for resolving the data dependencies caused by bucket shifting. As discussed in Section 5.1, there are two cases triggering bucket shifting during induced sorting, and two data races existing among the pipelined stages.

| Dependency | Strategy |
|---|---|
| Case 1 | Eliminated by initializing every bucket counter in $SA$ before the pipelined induced sorting. |
| Case 2 | Retained but only triggered within stage 3. |
| Race 1 | Eliminated by inducing on $R$ while updating on $SA$, and synchronizing the writes of all suffixes destined for a same bucket $BK_D$ to be done correctly among the threads before $BK_D$ shifting. |
| Race 2 | Eliminated by starting the stage 1 for $B_{k+1}$ only after the stage 3 for $B_{k-1}$ has been completed. |

$BK_D$ shifting triggered in stage 3. Moreover, stage 1 has no data dependency with the other two stages as both suffix updating and bucket shifting in stage 3 have been completed before stage 1 starts. Thus the data race 2 is solved as well and now three stages are safe to process the blocks in a pipeline as shown in Fig. 3b.

### 5.4 Algorithm

Algorithm 3 gives the details of the new pipeline at deeper levels. Before the pipelined sorting, line 1 initializes every bucket counter for the L-type suffixes in $SA$ in parallel, so that any bucket shifting must be triggered by case 2 only in stage 3. Both stage 1 and 3 in the pipeline are fully parallel, only stage 2 is sequential. Although the time complexity of pSACAK is $\mathcal{O}(n)$ due to the sequential stage, the other two parallel stages in the pipeline reduce the average memory access latency by utilizing the machine's parallel computing power, and hence yield a speedup over the sequential SACAK algorithm. Assume that the block size is $\beta$, and the whole $SA$ is evenly divided into $\lceil n/\beta \rceil$ blocks. The algorithm comprises three nodes in the pipeline as described below.

Parallel node of preparing for $B_{k+1}$: Lines 3-10 read the preceding character of each suffix currently in $B_{k+1}$ to produce a tuple $\langle chr, pos \rangle$ for any L-type preceding suffix in parallel using multiple threads. All the tuples are stored in the read buffer $R$. Lines 7-10 compute the correct location for each sorted suffix, and use it to store the corresponding tuple in $R$. The justification of a correct location is the existence of suffix $\mathsf{suf}(X, SA[i])$'s bucket counter in $SA[X[j]]$. If the counter does not exist as line 10, the correct location for $\langle chr, pos \rangle$ in $R$ is $\mathsf{suf}(X, SA[i])$'s position $i$ in $SA$ minus the block offset $(k+1) \cdot \beta$; otherwise, the correct location is $i - (k+1) \cdot \beta - 1$ due to the reused counter as line 8. Such a correct location for $\langle chr, pos \rangle$ in $R$ preserves the original position order of $\mathsf{suf}(X, SA[i])$ in $SA$, hence guarantees the correctness of the sorting process on $R$ in inducing node for $B_{k+1}$ at the next pipeline cycle. To avoid data racing, no bucket shifting in $SA$ is allowed during the preparing stage, thus stage 1 and 3 of the pipeline must not overlap.

Sequential node of inducing for $B_k$: Line 11 is a sequential loop, which scans $R$ to sort the L-type preceding suffixes of the suffixes sorted in $B_k$. For each scanned suffix, its preceding character is already cached in $R$, by either the preparing stage or the inducing stage. We can get the destination for the preceding suffix by accessing the bucket counter, and a tuple $\langle idx, pos \rangle$ for this suffix is stored to the write buffer $W$. If the correct location $idx - 1$ of this newly sorted suffix is inside $B_k$ or $B_{k+1}$, and its preceding suffix is L-type as well, another tuple $\langle chr_1, pos_1 \rangle$ must be put into the correct location in $R$ to be scanned later. In this case, the preceding character $chr_1$ has to be read from $X$ on-the-fly, but such random memory accesses are rare when $B_k$ and $B_{k+1}$ are small compared to $SA$. Moreover, the correct location of $\langle chr_1, pos_1 \rangle$ in $R$ is easier to be computed than that in lines 7-10, as the reused counter $SA[c]$ always exists during the current inducing stage, i.e., the destined bucket $BK_D$ is not yet fully filled.

**Algorithm 3:** Pipelined sorting L-type suffixes of $B_k$ at deeper levels.

**Input:** $X$: input string;
$SA$: suffix array with sorted LMS-suffixes;
**Output:** $SA$: suffix array with sorted L-type suffixes;

1 Scan $X$ once in parallel to initialize every bucket counter for L-type suffixes in $SA$ as $-1$;
/* Parallel Node: Preparing for $B_{k+1}$. */
2 Initialize each item of the read/write buffer $R[0, \beta-1]/W[0, \beta-1]$ as empty;
3 **parallel foreach** $SA[i] > 0$ *in* $B_{k+1}$ **do**
4 $j \leftarrow SA[i], pos \leftarrow j - 1$;
5 **if** $X[pos]$ *is L-type, i.e.,* $X[pos] \geq X[j]$ **then**
6 Read the preceding character by $chr \leftarrow X[pos]$;
7 **if** $SA[X[j]]$ *is being reused as bucket counter* **then**
8 Insert a tuple $\langle chr, pos \rangle$ to $R[i - (k+1) \cdot \beta - 1]$;
9 **else** // The correct location in $R$ is found.
10 Insert a tuple $\langle chr, pos \rangle$ to $R[i - (k+1) \cdot \beta]$;
/* Sequential Node: Inducing for $B_k$. */
11 **for** $i = 0$ **to** $\beta - 1$ **do**
12 **if** $R[i].pos \geq 0$ **then**
13 Get the preceding suffix by $pos \leftarrow R[i].pos$;
14 Get the start position of $BK_D$ by $c \leftarrow R[i].chr$;
15 Access the bucket counter by $d \leftarrow SA[c] - -$;
16 Get the destination in $SA$ by $idx \leftarrow c - d$;
17 Insert a tuple $\langle idx, pos \rangle$ to $W$;
18 **if** $idx - 1$ *belongs to* $B_k$ **or** $B_{k+1}$ **then**
19 **if** $pos > 0$ **and** $X[pos - 1]$ *is L-type* **then**
20 $pos_1 \leftarrow pos - 1, chr_1 \leftarrow X[pos_1]$;
21 Insert a tuple $\langle chr_1, pos_1 \rangle$ to the correct location in $R[idx - k \cdot \beta - 1]$;
/* Parallel Node: Updating for $B_{k-1}$. */
22 **parallel foreach** $\langle idx, pos \rangle$ *in* $W$ **do**
23 **if** $SA[idx]$ *is empty* **then**
24 $SA[idx] \leftarrow pos$;
25 **else** // Running into the start item of $BK_R$.
26 Synchronize the writes of all suffixes destined for $BK_D$ to be done correctly among the threads;
27 Shift left all items in $BK_D$ by one step;
28 $SA[idx - 1] \leftarrow pos$;
29 **return**

0018-9340 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TC.2018.2842050, IEEE Transactions on Computers



Parallel node of updating for $B_{k-1}$: Lines 22-28 write the suffixes in $W$ to their destinations in $SA$ using multiple threads by two steps as follows. (1) Write every suffix not destined for the end item of its bucket by setting $SA[idx] = pos$ in parallel. (2) Synchronize the writes of all suffixes destined for a same bucket to be done correctly among the threads, then each thread can shift left a full bucket in parallel and finally write the end item by setting $SA[idx-1] = pos$ in parallel too.

### 5.5 Example

Fig. 4 illustrates Algorithm 3 by a running example. Suppose the block size $\beta = 5$, and $SA$ is evenly divided into four blocks $\{B_0, B_1, B_2, B_3\}$ with only the last block being smaller. Initially, all the sorted LMS-suffixes of $X$ have been put into their buckets in $SA$ (i.e., $SA[0, 6, 7, 8] = \{16, 10, 6, 2\}$), and every bucket counter for L-type suffixes in $SA$ has been initialized as '$-1$' (i.e., $SA[1, 2, 9, 11, 13] = \{-1, -1, -1, -1, -1\}$). The algorithm begins with the preparing stage for $B_0$ at cycle 0. Then at cycle 1, the inducing stage for $B_0$ and the preparing stage for $B_1$ are processed simultaneously in the pipeline. When the algorithm approaches cycle 2, the inducing stage is going to process $B_1$, the corresponding read buffer is $R_2 = \{-, \langle 13, 9\rangle, \langle 13, 5\rangle, \langle 9, 1\rangle, -\}$ and the bucket counters are $SA[1, 2, 9, 11, 13] = \{-2, -2, -1, -2, -1\}$. Now, three stages of the pipeline run in parallel as follows.

Preparing stage for $B_2$: Once the updating stage for $B_0$ is complete at the step $i = 2$ of cycle 2, two threads $\tau_1$ and $\tau_2$ simultaneously read the preceding characters of the suffixes in $B_2$ and insert the tuples to $R_1$ for those L-type preceding suffixes. $\tau_1$ and $\tau_2$ handle the suffixes in $\{SA[10], SA[11]\}$ and $\{SA[12], SA[13], SA[14]\}$, respectively. $\tau_1$ reads nothing as $SA[10]$ is empty and $SA[11]$ is being reused as a bucket counter. At step $i = 2$, $\tau_2$ reads the preceding character of suffix in $SA[12] = 13$, which is $X[12] = 11$ and L-type, then inserts a tuple $\langle 11, 12\rangle$ to the correct location $R_1[2-1] = R_1[1]$. After that, $\tau_2$ reads nothing as $SA[13]$ is being reused as a bucket counter and $SA[14]$ is empty.

Inducing stage for $B_1$: A thread $\tau_3$ scans the L-type preceding suffixes in $R_2$ sequentially. At step $i = 0$, $R_2[0]$ is empty and skipped. At step $i = 1$, $R_2[1] = \langle 13, 9\rangle$ indicates that the preceding suffix of $SA[6] = 10$ is L-type and its destination in $SA$ is given by $13 - SA[13] = 13 + 1 = 14$, hence a tuple $\langle 14, 9\rangle$ is stored in $W_2[1]$ and $SA[13] = SA[13]-1 = -2$; as the correct location $idx-1 = 14-1 = 13$ of this suffix belongs to $B_2$, its preceding character is read to be $X[8] = 13$ and L-type, thus a tuple $\langle 13, 8\rangle$ is inserted to $R_1[3]$. At step $i = 2$, $R_2[2] = \langle 13, 5\rangle$ indicates that the preceding suffix of $SA[7] = 6$ is L-type and its destination in $SA$ is given by $13 - SA[13] = 13 + 2 = 15$, hence a tuple $\langle 15, 5\rangle$ is stored in $W_2[2]$ and $SA[13] = SA[13] - 1 = -3$; as the correct location $idx - 1 = 15 - 1 = 14$ of this suffix belongs to $B_2$, its preceding character is read to be $X[4] = 13$ and L-type, thus a tuple $\langle 13, 4\rangle$ is inserted to $R_1[4]$. At step $i = 3$, $R_2[3] = \langle 9, 1\rangle$ indicates that the preceding suffix of $SA[8] = 2$ is L-type and its destination in $SA$ is given by $9 - SA[9] = 9 + 1 = 10$, hence a tuple $\langle 10, 1\rangle$ is stored in $W_2[3]$ and $SA[9] = SA[9] - 1 = -2$; as the correct location $idx - 1 = 10 - 1 = 9$ of this suffix belongs to $B_1$, its preceding character is read to be $X[0] = 9$ and L-type, thus a tuple $\langle 9, 0\rangle$ is inserted to $R_2[4]$. At step $i = 4$, $R_2[4] = \langle 9, 0\rangle$ indicates that the preceding suffix of $SA[9] = 1$ is L-type and its destination in $SA$ is given by $9 - SA[9] = 9 + 2 = 11$, hence a tuple $\langle 11, 0\rangle$ is stored in $W_2[4]$ and $SA[9] = SA[9] - 1 = -3$.

Updating stage for $B_0$: Two threads $\tau_4$ and $\tau_5$ write the suffixes in $W_1$ to $SA$ in parallel. At steps $i = 0$ and $i = 1$, $\tau_4$ writes the suffixes $W_1[0] = \langle 2, 15\rangle$ and $W_1[1] = \langle 3, 14\rangle$ to $SA$ by setting $SA[2] = 15$ and $SA[3] = 14$, respectively. As $SA[2]$ is being reused as a counter of the right neighboring bucket, $\tau_4$ synchronizes among the threads to ensure no other suffix will be sorted into the destined bucket, then writes the last item of that bucket by setting $SA[1] = SA[2-1] = 15$. At steps $i = 0$, $\tau_5$ writes the suffix $W_1[2] = \langle 12, 13\rangle$ to $SA$ by setting $SA[12] = 13$. After that, $\tau_5$ writes nothing as both $W_1[3]$ and $W_1[4]$ are empty.

Repeating the same process for three more cycles will sort all the L-type suffixes in $SA$.

## 6 Experiments

Seven programs are evaluated in our experiments, i.e., pSACAK, pSAIS, pDSS, pKS, DSS, SAIS and SACAK. The first four are parallel and developed using Intel Cilk Plus [22], while the others are sequential. Herein, pSACAK, pSAIS, SAIS and SACAK were implemented by us for the algorithms proposed in this article, [12], [5] and [6], respectively. Every output SA produced by pSACAK or pSAIS was verified by the SA checker in the package of libdivsufsort [23]. While DSS is the sequential program of libdivsufsort which is based on induced copying/sorting, pDSS presented in [14] is a hybrid parallel algorithm of libdivsufsort and prefix doubling. pKS is a parallel implementation for the DC3 algorithm from the PBBS [19]. It is worth mentioning that the parallel implementation of the prefix doubling algorithm from PBBS is observed to run slower than pKS and use around $30n$ workspace on our machine, hence its performance results are not included.

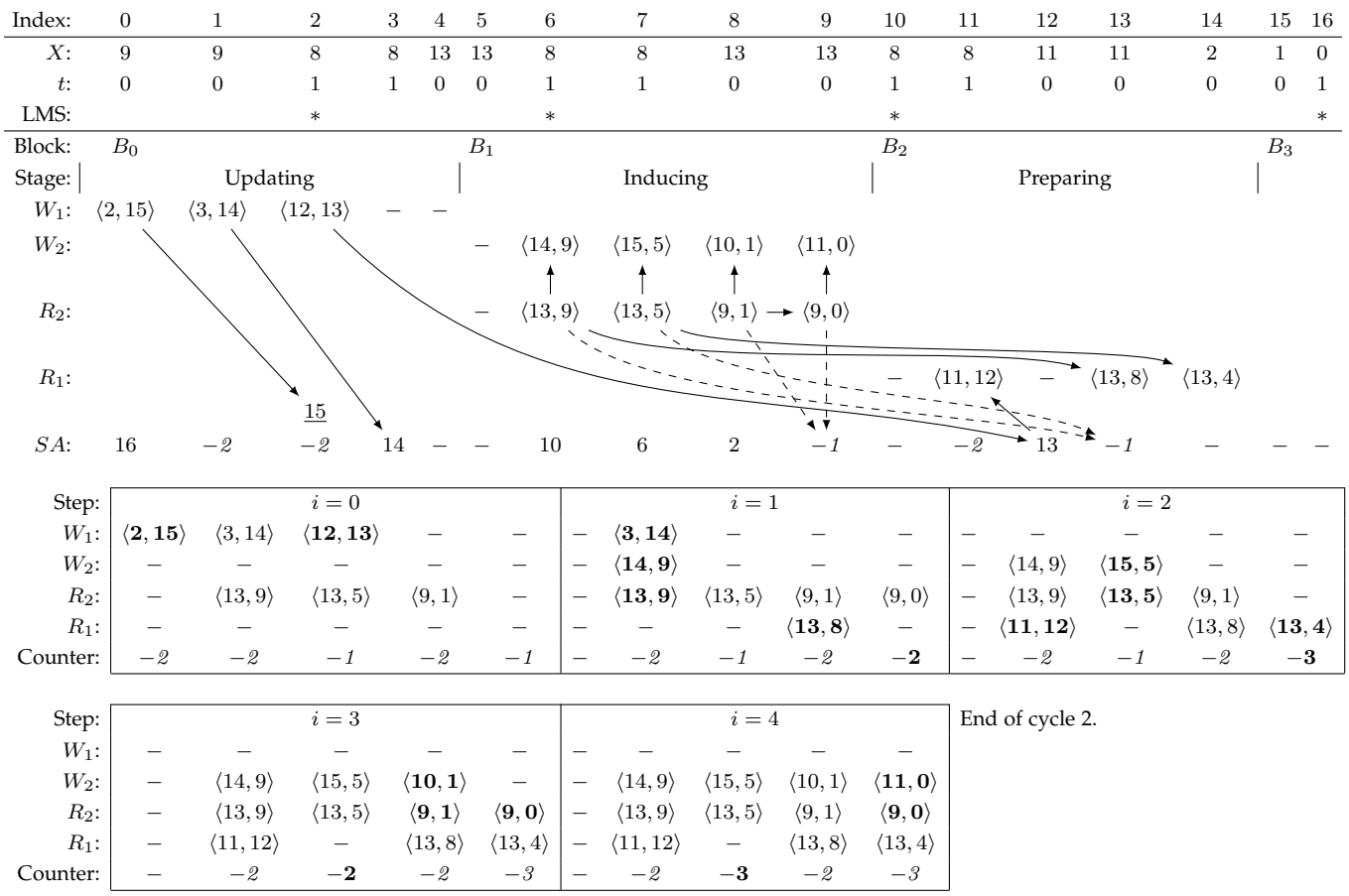
All the experiments were conducted on a machine with two Intel Xeon E5-2620 Processors (2.00 GHz, 15 MiB L3 cache, 6-core, 12 running threads via hyper-threading) and 64 GiB DDR3 RAM. The operating system was Linux (CentOS 6.8, 64-bit). The programs pSACAK and pSAIS were compiled by g++ version 5.3.1 with the options "-std=c++11 -O3 -DNDEBUG -fcilkplus", while the others were compiled with the default makefile provided in their source packages. Note that the experimental platform was the same as that in [12], but the pSAIS and pDSS programs in this article were implemented with Intel Cilk Plus instead of OpenMP or Intel TBB library for that in [12]. The performance of pSAIS and pDSS vary in this article and [12], due to the different parallel programming frameworks in use.

For comparison convenience, we use the same input datasets with [12] as shown in Table 2, which are a subset of the Pizza&Chili corpus [24] at http://pizzachili.dcc.uchile.cl. To test the programs on larger files, we include two additional datasets "proteins2" and "english2", such that the corpora varies from 200 MiB to 2 GiB. The former six are from the main text corpora and the latter four are from the highly repetitive corpora. The time and space

0018-9340 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TC.2018.2842050, IEEE Transactions on Computers






| Step: | $i=0$ | | | | | $i=1$ | | | | | $i=2$ | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $W_1$: | $\langle \mathbf{2,15} \rangle$ | $\langle 3,14 \rangle$ | $\langle \mathbf{12,13} \rangle$ | – | – | – | $\langle \mathbf{3,14} \rangle$ | – | – | – | – | – | – | – | – |
| $W_2$: | – | – | – | – | – | – | $\langle \mathbf{14,9} \rangle$ | – | – | – | – | $\langle 14,9 \rangle$ | $\langle \mathbf{15,5} \rangle$ | – | – |
| $R_2$: | – | $\langle 13,9 \rangle$ | $\langle 13,5 \rangle$ | $\langle 9,1 \rangle$ | – | – | $\langle \mathbf{13,9} \rangle$ | $\langle 13,5 \rangle$ | $\langle 9,1 \rangle$ | $\langle 9,0 \rangle$ | – | $\langle 13,9 \rangle$ | $\langle \mathbf{13,5} \rangle$ | $\langle 9,1 \rangle$ | – |
| $R_1$: | – | – | – | – | – | – | – | – | $\langle \mathbf{13,8} \rangle$ | – | – | $\langle \mathbf{11,12} \rangle$ | – | $\langle 13,8 \rangle$ | $\langle \mathbf{13,4} \rangle$ |
| Counter: | $-2$ | $-2$ | $-1$ | $-2$ | $-1$ | – | $-2$ | $-1$ | $-2$ | $\mathbf{-2}$ | – | $-2$ | $-1$ | $-2$ | $\mathbf{-3}$ |

| Step: | $i=3$ | | | | | $i=4$ | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| $W_1$: | – | – | – | – | – | – | – | – | – | – |
| $W_2$: | – | $\langle 14,9 \rangle$ | $\langle 15,5 \rangle$ | $\langle \mathbf{10,1} \rangle$ | – | – | $\langle 14,9 \rangle$ | $\langle 15,5 \rangle$ | $\langle 10,1 \rangle$ | $\langle \mathbf{11,0} \rangle$ |
| $R_2$: | – | $\langle 13,9 \rangle$ | $\langle 13,5 \rangle$ | $\langle \mathbf{9,1} \rangle$ | $\langle \mathbf{9,0} \rangle$ | – | $\langle 13,9 \rangle$ | $\langle 13,5 \rangle$ | $\langle 9,1 \rangle$ | $\langle \mathbf{9,0} \rangle$ |
| $R_1$: | – | $\langle 11,12 \rangle$ | – | $\langle 13,8 \rangle$ | $\langle 13,4 \rangle$ | – | $\langle 11,12 \rangle$ | – | $\langle 13,8 \rangle$ | $\langle 13,4 \rangle$ |
| Counter: | – | $-2$ | $\mathbf{-2}$ | $-2$ | $-3$ | – | $-2$ | $\mathbf{-3}$ | $-2$ | $-3$ |

End of cycle 2.

Fig. 4. A running example for the pipelined execution of Algorithm 3 at deeper levels. According to Property 4.1, each character in $X$ itself also points to its bucket counter in $SA$. The buffers $R_1$, $R_2$, $W_1$ and $W_2$ are only shown for cycle 2. Each tuple is $\langle chr, pos \rangle$ in $R$ and $\langle idx, pos \rangle$ in $W$. A solid arrow indicates a write operation, and the bucket shifting is triggered by writing the underlined suffix. A dashed arrow indicates the access to the suffix's bucket counter in $SA$. Each counter reuses the start item of its bucket in $SA$ and was initialized as '$-1$' before the pipelined sorting. Each block in $SA$ is marked only at its head position. An empty item in the mentioned data structures is marked as '$-$'.

TABLE 2
Corpus statistics, $n$ in Mi, one byte per character.

| Corpus | $n$ | $K$ | Description |
|---|---|---|---|
| proteins2 | 2048 | 27 | A concatenation of two copies of "proteins". |
| english2 | 2048 | 239 | A concatenation of two copies of "english". |
| proteins | 1024 | 27 | Protein sequences from Swissprot database. |
| english | 1024 | 239 | English text files from Gutenberg Project. |
| dna | 385 | 16 | DNA sequences from Gutenberg Project. |
| sources | 201 | 230 | C/Java source files of linux-2.6 and gcc-4.0. |
| einstein | 446 | 139 | The English article of Albert Einstein. |
| cere | 440 | 5 | The Saccharomyces Cerevisiae sequences. |
| fib41 | 256 | 2 | The Fibonacci sequence. |
| kernel | 246 | 160 | A collection of the Linux Kernel 6. |

consumptions of each program running on the datasets are measured for performance investigation. Specifically, since all the evaluated programs are 32-bit versions, each integer takes four bytes and each character of an input string takes one byte.

### 6.1 Space

The space usages measured in bytes per character for each program are listed in Table 3. Each space usage is the heap peak measured by shell command "memusage", which is maximum total space during the whole execution process. Each parallel program was allowed to run in full parallel on the machine, and the block size of pSACAK and pSAIS was set as $\beta = 100$Ki. Each mean is the average of all results for a same program, and the best results for the parallel algorithms are typeset in bold text. For comparison convenience, the normalized values are given in the last row.

Among the parallel programs, pSACAK is the only in-place one and always uses the least space much less than pDSS and pKS. Similar to pSAIS against SAIS, pSACAK uses a bit more space than SACAK due to the read/write buffers for the pipelined induced sorting. As the buffer size is fixed, the space results of pSACAK meet the theoretical analysis, i.e., $5n + \mathcal{O}(1)$ bytes, where $4n$ for the items in $SA$, $n$ for the input string and $\mathcal{O}(1)$ for the workspace.

In terms of the additional space caused by parallelization, the space ratios of pSACAK over SACAK, pSAIS over SAIS and pDSS over DSS are 1.004, 1.002 and 1.339, respectively. This indicates that both pSACAK and pSAIS have almost the same space requirements as their sequential

0018-9340 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TC.2018.2842050, IEEE Transactions on Computers



TABLE 3
Space usages (in bytes per character) and the incremental ratios of a parallel program over its sequential counterpart.

| Corpus | pSACAK | SACAK | | pSAIS | SAIS | | pDSS | DSS | | pKS |
|---|---|---|---|---|---|---|---|---|---|---|
| | $S_p$ | $S_s$ | $S_p/S_s$ | $S_p$ | $S_s$ | $S_p/S_s$ | $S_p$ | $S_s$ | $S_p/S_s$ | $S_p$ |
| proteins2 | **5.012** | 5.000 | 1.002 | 5.273 | 5.272 | 1.001 | 6.761 | 5.000 | 1.352 | 19.833 |
| english2 | **5.012** | 5.000 | 1.002 | 5.253 | 5.252 | 1.001 | 6.728 | 5.000 | 1.346 | 19.833 |
| proteins | **5.013** | 5.000 | 1.003 | 5.369 | 5.366 | 1.001 | 6.761 | 5.000 | 1.352 | 19.833 |
| english | **5.014** | 5.000 | 1.003 | 5.328 | 5.325 | 1.001 | 6.728 | 5.000 | 1.346 | 19.833 |
| dna | **5.019** | 5.000 | 1.004 | 5.376 | 5.367 | 1.002 | 6.531 | 5.001 | 1.306 | 19.833 |
| sources | **5.027** | 5.000 | 1.005 | 5.342 | 5.327 | 1.003 | 6.576 | 5.001 | 1.315 | 19.833 |
| einstein | **5.018** | 5.000 | 1.004 | 5.189 | 5.182 | 1.001 | 6.659 | 5.001 | 1.332 | 19.833 |
| cere | **5.018** | 5.000 | 1.004 | 5.186 | 5.179 | 1.001 | 6.434 | 5.001 | 1.287 | 19.833 |
| fib41 | **5.023** | 5.000 | 1.005 | 5.215 | 5.202 | 1.002 | 7.198 | 5.001 | 1.439 | 19.833 |
| kernel | **5.024** | 5.000 | 1.005 | 5.194 | 5.181 | 1.003 | 6.588 | 5.001 | 1.317 | 19.833 |
| Mean | **5.018** | 5.000 | 1.004 | 5.273 | 5.265 | 1.002 | 6.696 | 5.001 | 1.339 | 19.833 |
| Norm. | **1.004** | 1 | – | 1.055 | 1.053 | – | 1.339 | 1.001 | – | 3.967 |

counterparts, but pDSS requires around 1/3 more space than its sequential counterpart.

### 6.2 Time

In Table 4, each running time in seconds per MiB is the wall clock time measured by shell command "time", excluding the latency reported by the program for reading the input corpus and writing the output suffix array. Each result is the average of three runs, and the standard deviation is given to measure the performance fluctuation of each program. From the results in last three rows, we have the following observations.

- The best average running time is achieved by our program pSACAK. Specifically, its speedups over pSAIS and pDSS are 1.342 and 1.658, respectively. While pKS is about 5% slower than pSACAK, it requires much more space, i.e., up to 3.967/1.004=3.951.
- The slight fluctuations in running time of pSACAK, pSAIS and pKS are because these algorithms are linear-time with predictable performance. The super-linear algorithm pDSS runs fast on the upper four main text corpora, but suffers from performance degradations on the lower four highly repetitive corpora and those two larger files with an average LCP (Longest Common Prefix) of 1 GiB. The time performance of pDSS is sensitive to the input, and such a phenomenon has also been observed in [12].
- For the speedup of a parallel program over its sequential counterpart, there are pSACAK:SACAK =4.380, pSAIS:SAIS=4.094 and pDSS:DSS=1.718. This suggests that our parallelizations of the IS method in pSACAK and pSAIS are more efficient than that of pDSS. Specifically, the speedup of pSACAK is greater than pSAIS's by 4.380/4.094=1.070. This is mainly contributed by the parallelization at deeper levels, where pSACAK dynamically reuses some items of $SA$ as bucket counters and avoids the parallel scanning of $X$ up to six times for generating the bucket counter array in pSAIS.

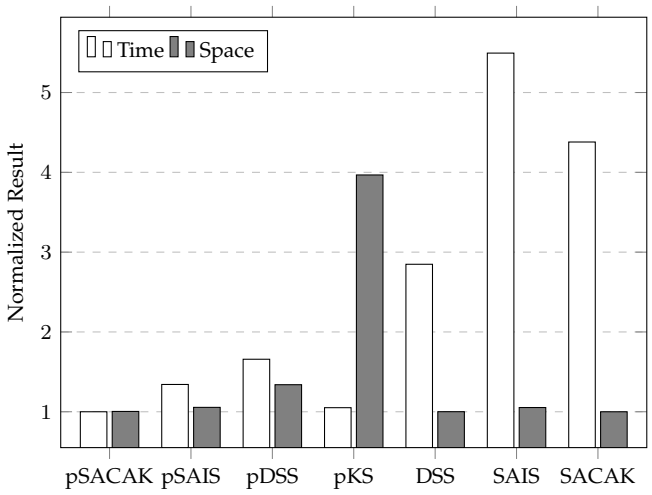



Fig. 5. The normalized running time and space usage of each program.

- For an overall comparison, Fig. 5 shows the normalized running time and space usage of each program. Compared to pSAIS, pSACAK runs 34% faster and uses 5% less space, by using no bucket counter array $bkt$ on deeper levels nor type array $t$ at all. As a conclusion of our experiment results, among all the programs in comparison, pSACAK achieves the best overall time and space performance.

### 6.3 Scalability

In order to investigate the scalability of these parallel programs with increasing input size, we ran pSACAK, pSAIS, pDSS and pKS on the increasing prefixes of corpora "english" and "einstein". Fig. 6 shows the running time results.

In both figures, a consistent trend for the curves of pKS indicates that its program slows down when the input size $n$ increases. The reason is that, the size of each run of suffix sorting in pKS is linear to $n$ and a larger $n$ leads to a higher on-chip cache miss ratio, which in turn degrades the overall performance of parallel sorting. However, as

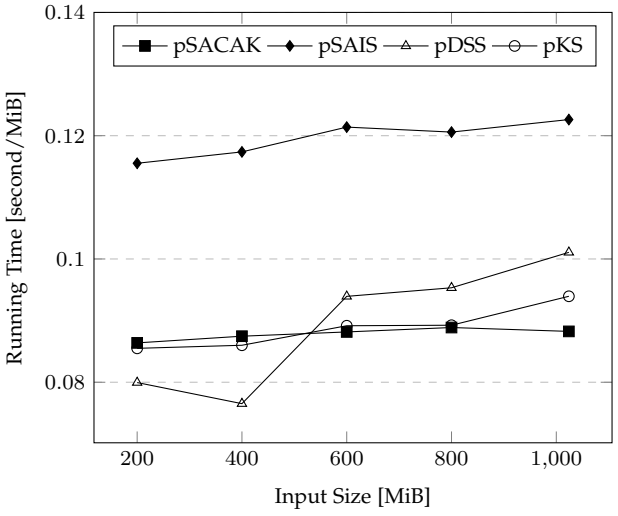
0018-9340 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TC.2018.2842050, IEEE Transactions on Computers

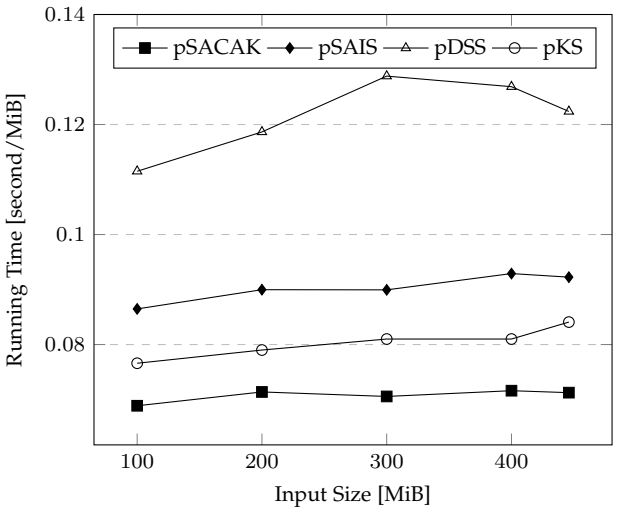


TABLE 4
Running time (in seconds per MiB) and the speedups of a parallel program over its sequential counterpart.

| Corpus | pSACAK | SACAK | | pSAIS | SAIS | | pDSS | DSS | | pKS |
|---|---|---|---|---|---|---|---|---|---|---|
| | $T_p$ | $T_s$ | $T_s/T_p$ | $T_p$ | $T_s$ | $T_s/T_p$ | $T_p$ | $T_s$ | $T_s/T_p$ | $T_p$ |
| proteins2 | 0.087 | 0.455 | 5.230 | 0.140 | 0.636 | 4.543 | 0.217 | 0.430 | 1.982 | **0.084** |
| english2 | 0.083 | 0.477 | 5.747 | 0.131 | 0.654 | 4.992 | 0.220 | 0.439 | 1.995 | **0.078** |
| proteins | 0.094 | 0.378 | 4.021 | 0.132 | 0.510 | 3.864 | **0.081** | 0.205 | 2.531 | 0.088 |
| english | **0.088** | 0.399 | 4.534 | 0.123 | 0.523 | 4.252 | 0.101 | 0.195 | 1.931 | 0.094 |
| dna | 0.088 | 0.351 | 3.989 | 0.109 | 0.419 | 3.844 | **0.063** | 0.181 | 2.873 | 0.084 |
| sources | 0.077 | 0.226 | 2.935 | 0.090 | 0.256 | 2.844 | **0.051** | 0.116 | 2.275 | 0.087 |
| einstein | **0.071** | 0.219 | 3.085 | 0.092 | 0.236 | 2.565 | 0.122 | 0.135 | 1.107 | 0.084 |
| cere | **0.075** | 0.331 | 4.413 | 0.094 | 0.405 | 4.309 | 0.114 | 0.160 | 1.404 | 0.082 |
| fib41 | 0.061 | 0.342 | 5.607 | 0.062 | 0.366 | 5.903 | 0.217 | 0.257 | 1.184 | **0.059** |
| kernel | **0.070** | 0.278 | 3.971 | 0.085 | 0.331 | 3.894 | 0.120 | 0.136 | 1.133 | 0.085 |
| Mean | **0.079** | 0.346 | 4.380 | 0.106 | 0.434 | 4.094 | 0.131 | 0.225 | 1.718 | 0.083 |
| Norm. | **1** | 4.380 | – | 1.342 | 5.494 | – | 1.658 | 2.848 | – | 1.051 |
| Stdev | 0.010 | 0.087 | – | 0.025 | 0.145 | – | 0.065 | 0.117 | – | **0.009** |




(a) english




(b) einstein

Fig. 6. Running time of pSACAK, pSAIS, pDSS and pKS for the increasing prefixes of corpora "english" and "einstein".

the block numbers in the pipelined induced sorting are linear to $n$ as well, pSACAK and pSAIS overcome the issue of increasing cache miss ratio by higher parallelism with more blocks in the pipeline. As a result, their curves in both figures have relatively stable trends. Being a hybrid of induced sorting and prefix doubling, the running time of pDSS is unpredictable for different input size, thus its curves are fluctuating in both figures.

Comparing the results in two figures, pDSS runs slower on the highly repetitive corpus "einstein". Although pDSS separates the input string into variable-size segments and uses multiple threads to run the segmented suffix sorting in parallel, the highly repetitive input string causes fewer segments for separation and hence exposes less parallelism, resulting in a longer running time. Furthermore, its time complexity is still depending on the longest run of any repeated character in the input string, which is $\mathcal{O}(n)$ in the worst case. On the other hand, pKS achieves a slight speedup on "einstein" over the main text corpus "english". For pKS, a more repetitive input string leads to a greater recursion depth but less iterations in each recursion level, and the latter benefits the running time and is dominating. Different from pKS with a reduction ratio of $2/3$, both pSACAK and pSAIS have a reduction ratio not greater than $1/2$. Such a distinct advantage yields their speedups better than that of pKS.

### 6.4 Efficiency

To see the efficiency of pSACAK and estimate the margin for further speedup on our 12-core machine, we cut the 1024 MiB dataset "english" into $\theta = \{1, 2, 4, 8, 16\}$ pieces and evaluated the mean runtime $T_m$ for running $\theta$ processes of SACAK program simultaneously, one piece per process. The metric $T_m$ gives a rough estimation of the best runtime for problem dividing by parallelizing SACAK on the machine. Moreover, the time for merging the output of each process to produce the final suffix array is roughly of the equivalent time complexity as that for problem dividing [25], [26], resulting in $2T_m$ as the theoretically minimum runtime for a

0018-9340 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TC.2018.2842050, IEEE Transactions on Computers



TABLE 5
The time results of SACAK for varying $\theta$ and the efficiency ratios of pSACAK. The input size per process is in MiB, the time results of SACAK are in seconds, and the efficiency ratios are in percentage.

| | | | | | |
|---|---|---|---|---|---|
| Multiple SACAK processes: | | | | | |
| $\theta$ | 1 | 2 | 4 | 8 | 16 |
| $n/\theta$ | 1024 | 512 | 256 | 128 | 64 |
| $T_m$ | 408 | 185 | 87 | 43 | 27 |
| $T_{est}$ | 408 | 463 | 218 | 108 | 68 |
| Single pSACAK process: | | | | | |
| $T_{est}/T_p$ | 453 | 514 | 242 | 120 | 76 |

parallel variant of SACAK. In order to be more reasonable, by counting the overhead of parallel processing, $0.5T_m$ more time is assumed. Hence for $\theta > 1$, $T_{est} = 2.5T_m$ in total is taken as the best runtime for any parallel variant of SACAK on our machine, and $T_{est}/T_p$ gives an estimation for how efficient pSACAK is in comparison with the best possible result, where $T_p = 90$ seconds is the runtime for a single pSACAK process on the whole 1024 MiB dataset "english".

Table 5 lists the experimental results. Given 12 CPU cores, $T_m$ increases significantly when $\theta > 12$, because the SACAK processes are slowed down by the increased competitions for computing resources. The maximum $\theta$ is chosen as 16, in this case, $T_{est}/T_p = 76\%$ indicates that the speed of pSACAK is 76% of the estimated fastest speed for any parallel variant of SACAK on this machine. In other words, if there is any better parallel variant than pSACAK, the margin for further speedup is only about 24%. Such a small margin suggests that pSACAK has efficiently exploited the machine's parallel computing power.

## 7 Conclusions

We have presented the pSACAK algorithm for the parallelization of induced sorting suffixes in $\mathcal{O}(n)$ time and $\mathcal{O}(K)$ workspace on a multicore machine. To the best of our knowledge, for a multicore machine, it is the only linear-time and in-place parallel suffix sorting algorithm for constant alphabets reported so far. Since our recent work pSAIS has exploited the parallelism of IS method for efficient memory accesses using multiple threads on a multicore machine, the focus here for designing pSACAK from SACAK is the pipelined induced sorting within the optimal $\mathcal{O}(K)$ workspace. A number of new techniques have been proposed to overcome the difficulties in the algorithm's design. Our experimental results indicate that this new parallel algorithm is the most time and space efficient among all in comparison.

pSACAK is a parallel variant of the sequential SACAK, saying that both have a same algorithmic framework and the former is obtained by parallelizing each step of the latter. As a result, pSACAK not only keeps the linear-time and in-place merits of SACAK, but also runs several times faster by exploiting the parallel computing power of a multicore machine. While the number of cores and the memory bandwidth of a modern computer are consistently increased, there is a demand in practice for solutions capable of fully utilizing the parallelism of more and more computing resources provided by the machine. This work provides a solution to meet the demand for time and space efficient suffix sorting on a multicore machine.

For sequential suffix sorting, the experimental results in [27] suggest that the linear-time algorithms were not fast, i.e., slower than the other super-linear ones. Later on, this speed gap was narrowed by SAIS and then further narrowed by SACAK. Although SACAK is linear-time and in-place, its program is observed to run slower than the super-linear program libdivsufsort in most cases for realistic inputs [6], because the latter is more cache-friendly. However, from our experiments for pSACAK here and that for pSAIS in [12], on a multicore machine, the parallelized IS algorithms have not only the theoretically optimal time and space complexities, but also the best practical time and space performance among all in comparison. By using multiple threads to reduce the average delay for data accesses during the IS procedures, both pSACAK and pSAIS run faster than pDSS (the parallel variant of libdivsufsort). This shows that on a multicore computer, suffix sorting can be linear-time, fast and in-place.

## Acknowledgments

The work of B. Lao was supported by the Guangzhou Science and Technology Program grant 201707010165. The work of G. Nong was supported by the Guangdong Science and Technology Program grant 2017A050506025. The work of W. H. Chan was supported by the General Research Fund (18300215) from the Research Grant Council of the University Grants Committee of Hong Kong SAR. The work of J. Y. Xie was supported by the Opening Project of Guangdong Province Key Laboratory of Information Security Technology (2017B030314131).

## References

[1] M. I. Abouelhoda, S. Kurtz, and E. Ohlebusch, "Replacing suffix trees with enhanced suffix arrays," *Journal of Discrete Algorithms*, vol. 2, no. 1, pp. 53–86, 2004.

[2] M. Burrows and D. J. Wheeler, "A block-sorting lossless data compression algorithm," *SRC Research Report*, vol. 124, pp. 1–24, 1994.

[3] A. M. S. Shrestha, M. C. Frith, and P. Horton, "A bioinformatician's guide to the forefront of suffix array construction algorithms," *Briefings in Bioinformatics*, vol. 15, no. 2, pp. 138–154, 2014.

[4] U. Manber and G. Myers, "Suffix arrays: a new method for on-line string searches," *SIAM Journal on Computing*, vol. 22, no. 5, pp. 935–948, 1993.

[5] G. Nong, S. Zhang, and W. H. Chan, "Two efficient algorithms for linear time suffix array construction," *IEEE Transactions on Computers*, vol. 60, no. 10, pp. 1471–1484, 2011.

[6] G. Nong, "Practical linear-time O(1)-workspace suffix sorting for constant alphabets," *ACM Transactions on Information Systems*, vol. 31, no. 3, pp. 15:1–15:15, 2013.

[7] G. Nong, W. H. Chan, S. Zhang, and X. F. Guan, "Suffix array construction in external memory using d-critical substrings," *ACM Transactions on Information Systems*, vol. 32, no. 1, pp. 1:1–1:15, 2014.

[8] G. Nong, W. H. Chan, S. Q. Hu, and Y. Wu, "Induced sorting suffixes in external memory," *ACM Transactions on Information Systems*, vol. 33, no. 3, pp. 12:1–12:15, 2015.

[9] V. Osipov, "Parallel suffix array construction for shared memory architectures," in *Proceedings of the 19th International Symposium on String Processing and Information Retrieval*. Springer, 2012, pp. 379–384.

0018-9340 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TC.2018.2842050, IEEE Transactions on Computers




[10] M. Deo and S. Keely, "Parallel suffix array and least common prefix for the GPU," in *Proceedings of the 18th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*. ACM, 2013, pp. 197–206.

[11] L. Wang, S. Baxter, and J. D. Owens, "Fast parallel skew and prefix-doubling suffix array construction on the GPU," *Concurrency and Computation: Practice and Experience*, vol. 28, no. 12, pp. 3466–3484, 2016.

[12] B. Lao, G. Nong, W. H. Chan, and Y. Pan, "Fast induced sorting suffixes on a multicore machine," submitted for publication.

[13] W. Sun, "Using GPU to accelerate suffix array construction," in *Proceedings of the 7th International Conference on BioMedical Engineering and Informatics*. IEEE, 2014, pp. 677–682.

[14] J. Labeit, J. Shun, and G. E. Blelloch, "Parallel lightweight wavelet tree, suffix array and FM-index construction," *Journal of Discrete Algorithms*, vol. 43, no. 1, pp. 2–17, 2017.

[15] S. J. Puglisi, W. F. Smyth, and A. H. Turpin, "A taxonomy of suffix array construction algorithms," *ACM Computing Surveys*, vol. 39, no. 2, pp. 4:1–4:31, 2007.

[16] J. Kärkkäinen, D. Kempa, S. J. Puglisi, and B. Zhukova, "Engineering external memory induced suffix sorting," in *Proceedings of the 19th Workshop on Algorithm Engineering and Experiments*. SIAM, 2017, pp. 98–108.

[17] J. Kärkkäinen, P. Sanders, and S. Burkhardt, "Linear work suffix array construction," *Journal of the ACM*, vol. 53, no. 6, pp. 918–936, 2006.

[18] G. Franceschini and S. Muthukrishnan, "In-place suffix sorting," in *Proceedings of the 34th International Conference on Automata, Languages and Programming*. Springer, 2007, pp. 533–545.

[19] J. Shun *et al.*, "Brief announcement: the problem based benchmark suite," in *Proceedings of the Twenty-fourth Annual ACM Symposium on Parallelism in Algorithms and Architectures*. ACM, 2012, pp. 68–70.

[20] N. J. Larsson and K. Sadakane, "Faster suffix sorting," *Theoretical Computer Science*, vol. 387, no. 3, pp. 258–272, 2007.

[21] J. Kärkkäinen and P. Sanders, "Simple linear work suffix array construction," in *Proceedings of the 30th International Conference on Automata, Languages and Programming*. Springer, 2003, pp. 943–955.

[22] Intel Corporation. (2017) Intel Cilk Plus, an extension to the C and C++ languages to support data and task parallelism. [Online]. Available: https://www.cilkplus.org

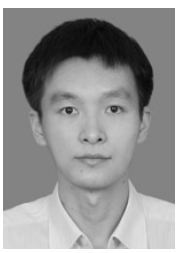
[23] Y. Mori. (2015) Libdivsufsort, a software library that implements a lightweight suffix array construction algorithm. [Online]. Available: https://github.com/y-256/libdivsufsort

[24] P. Ferragina and G. Navarro. (2005) Pizza&Chili Corpus, compressed indexes and their testbeds. [Online]. Available: http://pizzachili.dcc.uchile.cl

[25] J. Fischer, D. Köppl, and F. Kurpicz, "On the benefit of merging suffix array intervals for parallel pattern matching," in *Proceedings of the 27th Annual Symposium on Combinatorial Pattern Matching*. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2016, pp. 26:1–26:11.

[26] L. Egidi and G. Manzini, "Lightweight BWT and LCP merging via the gap algorithm," in *Proceedings of the 24th International Symposium on String Processing and Information Retrieval*. Springer, 2017, pp. 176–190.

[27] A. Antonitio, P. Ryan, B. Smyth, A. Turpin, and X. Yu, "New suffix array algorithms - linear but not fast?" in *Proceedings of the 15th Australasian Workshop on Combinatorial Algorithms*. Australian Computer Society, 2004, pp. 148–156.

**Bin Lao** received the bachelor's degree from South China Normal University in 2008, and the master's degree from Sun Yat-sen University in 2012, both in China. He is now a PhD candidate in the Department of Computer Science, Sun Yat-sen University, China. His current research interests include data processing and parallel computing.



**Ge Nong** received the BE and ME degrees in computer engineering from the NanJing University of Aeronautics and Astronautics in 1992, and the South China University of Science and Technology in 1995, respectively. He received the PhD degree in computer science from the Hong Kong University of Science and Technology in 1999. Then, he joined STMicroelectronics as a senior researcher with R&D on IC and system technologies for high-speed switches and routers. He is now a professor in the Department of Computer Science of Sun Yat-sen University in Guangzhou, China. His current research interests include algorithms, computer and communication networks, switching theory, and performance evaluation. He is a member of the IEEE.



**Wai Hong Chan** received the BSc, MPhil and PhD in Mathematical Science from the Hong Kong Baptist University, Hong Kong Special Administrative Region, China, in 1994, 1996 and 2003 respectively. He joined the Department of Mathematics and Information Technology of the Education University of Hong Kong in 2011, and now the Head of Department. His current research interests include algorithm design, quantum information, graph theory and combinatorics.



**Jing Yi Xie** received the BE degree in software engineering from Sun Yat-sen University, China, in 2017. She is currently working towards the ME degree in software engineering at Sun Yat-sen University, China. Her current research interest is string processing for massive data.

0018-9340 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.